Lab 7: Fuzzing for Input Validation Bugs

Ryona Aditree Ghosh | PES1UG24AM233 | AIML-D

Property-based fuzz testing of `self_storage.py` with Hypothesis

Repository: github.com/ryonahh/L6_L7_51_self_storage (Lab 7)

1. Objective

To explore fuzz testing as a security and quality-assurance technique by finding and fixing input-validation bugs in the provided Python module using the Hypothesis library. The module parses storage rentals, averages rental durations and computes the payable rent after discount. Three functions are under test: `parse_rental`, `duration_average` and `payable_rent`.

2. Tools and Environment

Item	Version / detail
Operating system	Windows 11 (win32)
Python	3.11.9
pytest	9.2.0
Hypothesis	6.170.15
Code under test	self_storage.py (Lab 7)

3. Methodology

Fuzzing feeds random, unexpected or malformed data to a program to expose crashes and incorrect behaviour. Instead of hand-written cases, Hypothesis lets us state *properties* that must hold for every input and then generates hundreds of inputs per test (text, None, integers, floats, lists), shrinking any failure to a minimal example. Thirteen tests were written in `test_self_storage.py`:

Property tested	Functions	Tests
Never crashes for any text or None (the handout's property)	all three	3
Never crashes for any type: text, None, ints, floats, lists	all three	3
Valid input round-trips to the expected value	parse_rental	1
Result is clean: non-empty unit, finite positive months	parse_rental	1
Average lies between min and max, and equals the true mean	duration_average	2
Rent is finite and never negative; result is clean	payable_rent	2
Known example values (medium:6, 3,6,12, 2400|200)	all three	1

4. Results on the Original (Buggy) Code

4.1 Running the application (Step 2)

With well-formed input the original program behaves correctly (Figure 1). With a malformed rental string it terminates with an unhandled exception (Figure 2).

```
C:\Users\ryona\L6_L7_51_self_storage\Lab 7>python self_storage.py
==== Running parse_rental ====
Enter unit:months (e.g. medium:6): medium:6
Unit and Months: ('Medium', 6.0)

==== Running duration_average ====
Enter rental months as CSV (e.g. 3,6,12): 3,6,12
Average Duration: 7

==== Running payable_rent ====
Enter rent|discount (e.g. 2400|200): 2400|200
Payable Rent: 2632.0
```

Figure 1: Original application, valid input (medium:6 / 3,6,12 / 2400|200).

```
C:\Users\ryona\L6_L7_51_self_storage\Lab 7>python self_storage.py
==== Running parse_rental ====
Enter unit:months (e.g. medium:6): medium,
Traceback (most recent call last):
  File "C:\Users\ryona\L6_L7_51_self_storage\Lab 7\self_storage.py", line 31, in <module>
    main()
  File "C:\Users\ryona\L6_L7_51_self_storage\Lab 7\self_storage.py", line 17, in main
    result = parse_rental(user_input)
             ^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\ryona\L6_L7_51_self_storage\Lab 7\self_storage.py", line 2, in parse_rental
    unit, months = text.split(':')
    ^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

Figure 2: Original application crashes on input without ':' (ValueError).

4.2 Fuzz tests against the buggy code (Step 4)

Running `python -m pytest test_self_storage.py` gave **9 failed, 4 passed** (Figure 3). Hypothesis reduced each failure to a minimal counter-example.

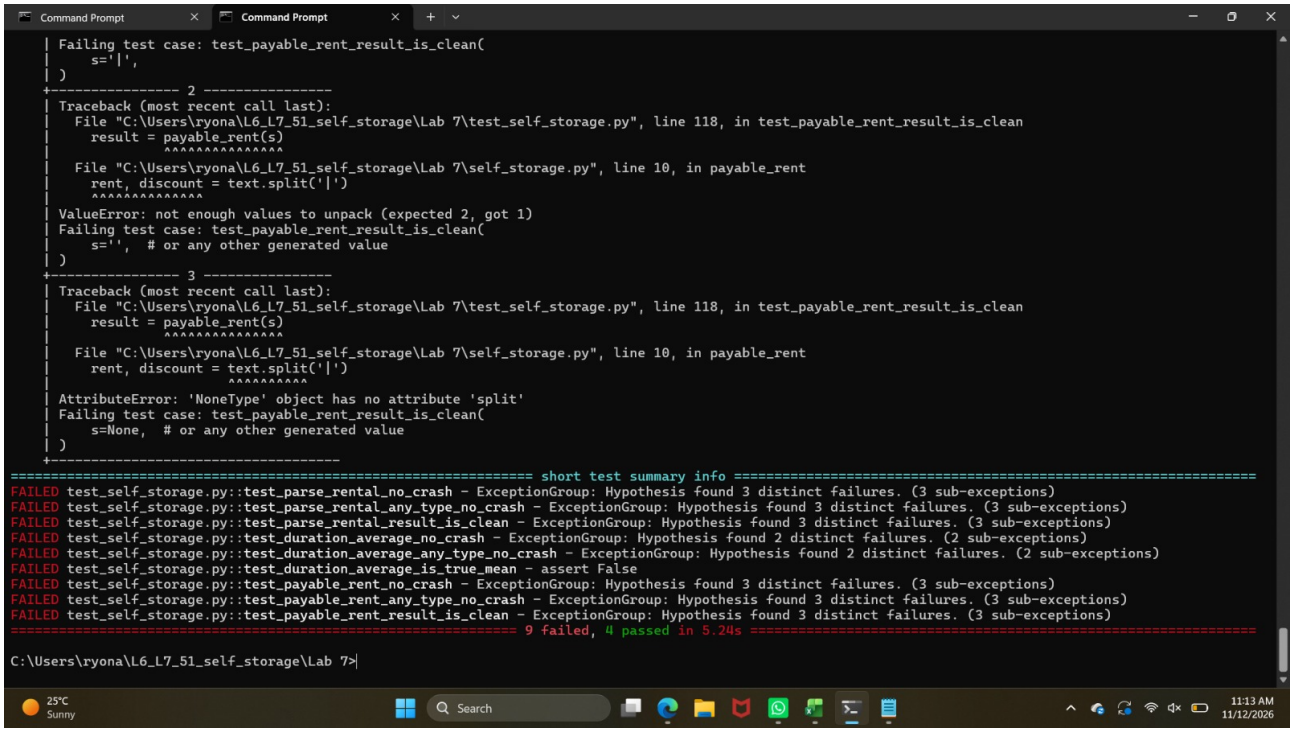


Figure 3: pytest + Hypothesis on the original code: 9 failed, 4 passed.

4.3 Analysis of the bugs found

Function	Failing input	Original behaviour	Root cause
parse_rental	None	AttributeError	No type check before .split()
parse_rental	'' (empty)	ValueError: not enough values to unpack	Assumes exactly one ':'
parse_rental	':'	ValueError: could not convert '' to float	Empty unit and months not validated
parse_rental	'a:-5', 'a:nan'	Accepted silently	Non-positive and non-finite months allowed
duration_average	None	AttributeError	No type check
duration_average	'' (empty)	ValueError: invalid literal for int()	Empty element not handled
duration_average	'3,4'	Returns 3 (true mean 3.5)	Floor division (//) truncates; negatives accepted
payable_rent	None	AttributeError	No type check
payable_rent	'' and '|'	ValueError	Assumes exactly one '|'; empty fields
payable_rent	'nan|1', 'inf|1'	Returns nan / inf	float() accepts nan and inf
payable_rent	'-100|0'	Returns 0 silently	Negative rent accepted

5. Fixes Implemented

Each function was hardened with defensive checks. Invalid input now returns `None` and `main()` prints a clear message instead of a traceback. The key changes were:

• **Type check**: reject anything that is not a string.
• **Structure check**: require exactly one ':' or '|' separator, and non-empty fields.
• **Safe parsing**: wrap float()/int() in try/except and accept only finite values (rejects nan, inf, overflow).
• **Range checks**: months must be positive; rent and discount must be non-negative.
• **Correct maths**: duration_average returns a true mean (sum / len) instead of floor division; payable_rent rounds the result to 2 decimals.

```
def _to_finite_float(value):
    try:
        number = float(value)
    except (TypeError, ValueError, OverflowError):
        return None
    return number if math.isfinite(number) else None

def parse_rental(text):
    if not isinstance(text, str):
        return None
    parts = text.split(':')
    if len(parts) != 2:
        return None
    unit = parts[0].strip()
    months = _to_finite_float(parts[1].strip())
    if not unit or months is None or months <= 0:
        return None
    return unit.title(), months
```

6. Verification of the Fixed Code

6.1 Running the fixed application (Step 5)

Malformed inputs (`medium,`, `3,,12`, `2400|-5`) now produce clear messages and no traceback (Figure 4).

```
C:\Users\ryona\L6_L7_51_self_storage\Lab 7>python self_storage.py
==== Running parse_rental ====
Enter unit:months (e.g. medium:6): medium,
Unit and Months: Invalid input (expected unit:months with months > 0)

==== Running duration_average ====
Enter rental months as CSV (e.g. 3,6,12): 3,,12
Average Duration: Invalid input (expected positive whole numbers, comma-separated)

==== Running payable_rent ====
Enter rent|discount (e.g. 2400|200): 2400|-5
Payable Rent: Invalid input (expected non-negative rent|discount)
```

Figure 4: Fixed application on malformed input: friendly messages, no crash.

6.2 Re-running the fuzz tests (Step 6)

All thirteen tests pass against the fixed code (Figure 5).

```
C:\Users\ryona\L6_L7_51_self_storage\Lab 7>python -m pytest test_self_storage.py -v
============================= test session starts =============================
platform win32 -- Python 3.11.9, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\ryona\AppData\Local\Programs\Python\Python311\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\ryona\L6_L7_51_self_storage\Lab 7
plugins: anyio-4.15.1, hypothesis-6.170.15
collected 13 items

test_self_storage.py::test_parse_rental_no_crash PASSED                  [  7%]
test_self_storage.py::test_parse_rental_any_type_no_crash PASSED         [ 15%]
test_self_storage.py::test_parse_rental_valid_roundtrip PASSED           [ 23%]
test_self_storage.py::test_parse_rental_result_is_clean PASSED           [ 30%]
test_self_storage.py::test_duration_average_no_crash PASSED              [ 38%]
test_self_storage.py::test_duration_average_any_type_no_crash PASSED     [ 46%]
test_self_storage.py::test_duration_average_between_min_and_max PASSED   [ 53%]
test_self_storage.py::test_duration_average_is_true_mean PASSED          [ 61%]
test_self_storage.py::test_payable_rent_no_crash PASSED                  [ 69%]
test_self_storage.py::test_payable_rent_any_type_no_crash PASSED         [ 76%]
test_self_storage.py::test_payable_rent_never_negative_and_finite PASSED [ 84%]
test_self_storage.py::test_payable_rent_result_is_clean PASSED           [ 92%]
test_self_storage.py::test_known_values PASSED                           [100%]

============================= 13 passed in 1.05s ==============================
```

Figure 5: pytest + Hypothesis on the fixed code: 13 passed.

7. Conclusion

Fuzzing exposed bugs that a handful of hand-picked examples would have missed: crashes on None, empty and malformed strings, silent acceptance of nan, inf and negative values, and an incorrect average caused by floor division. After adding validation, every function handles arbitrary input without raising, and all 13 property tests pass. The original program crashed on bad input; the fixed program reports it cleanly.

Behaviour changes to note: invalid input now returns None rather than raising, duration_average returns a true mean (e.g. 7.0 instead of 7), and payable_rent rounds to 2 decimals. **Limitation:** Hypothesis samples inputs randomly, so a passing run increases confidence but does not prove the absence of bugs.